ArUco marker e OpenCV

Fondamenti di Robotica

Docenti: Loredana Zollo
Clemente Lauretti

Tutor: **Rita Molle**
email: rita.molle@unicampus.it



UNIVERSITA'
CAMPUS
BIO-MEDICO
DI ROMA



CAMPUS BIO-MEDICO UNIVERSITY OF ROME
Via Álvaro del Portillo, 21 - 00128 Rome - Italy
www.unicampus.it

Outline

- Object detection
- Cosa sono gli ArUco Marker
- Tecnica di codifica
- Benefici degli ArUco
- Generare gli ArUco Marker – OpenCV
- Individuare il marker – OpenCV
- Calibrazione della camera
- Individuare il marker – OpenCV & ROS2

Object detection

Tecnica che permette a un sistema di **individuare e localizzare** uno o più oggetti all'interno di un'immagine o di un video

Markerless

Riconosce gli oggetti con feature visive

- Nessun marker fisico
- Complesso da implementare
- Sensibile a luce e angolo
- Richiede AI
- Stima della posa complessa
- Ambienti non strutturati

Marker-based

Riconosce gli oggetti con marker artificiali

- Uso di marker stampati
- Facile e veloce da integrare
- Robusto e accurato
- Usa librerie esistenti
- Stima della posa veloce
- Ambienti strutturati

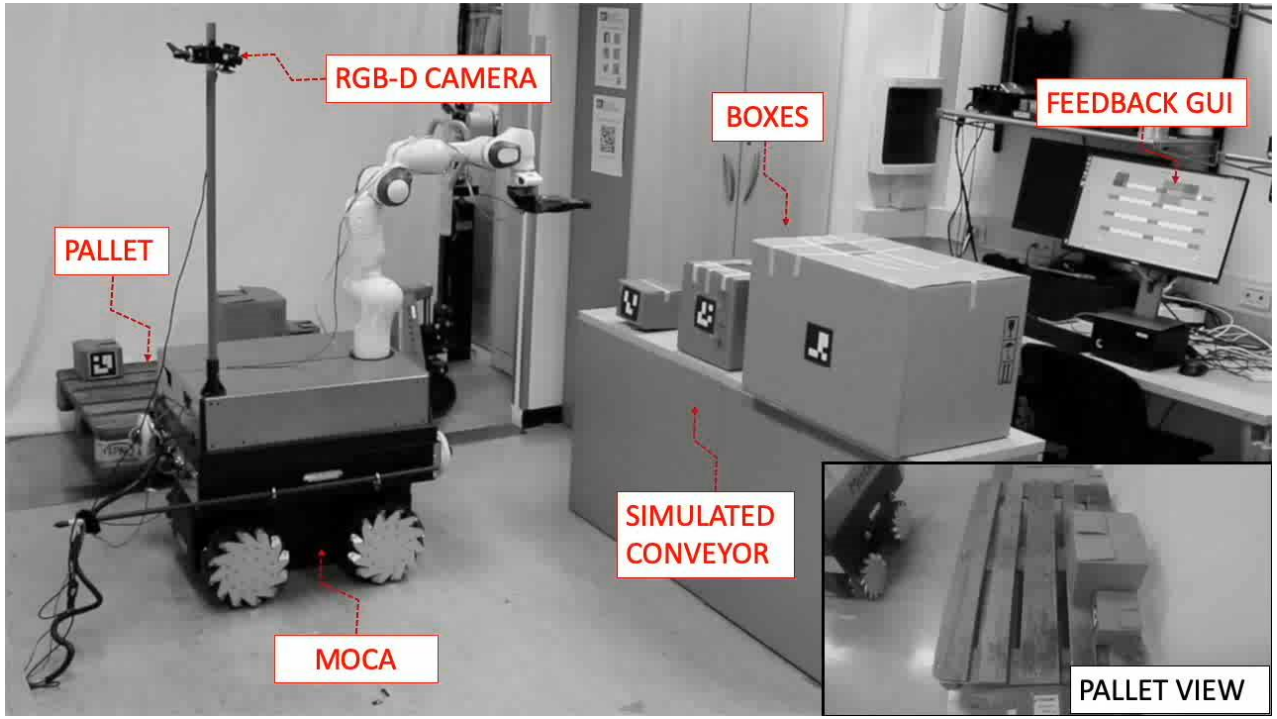
Object detection

ArUco marker

Affidabile e accurato

Rilevamento in tempo reale

No addestramento di modelli



Applicazioni: realtà aumentata, robotica, tracking, navigazione

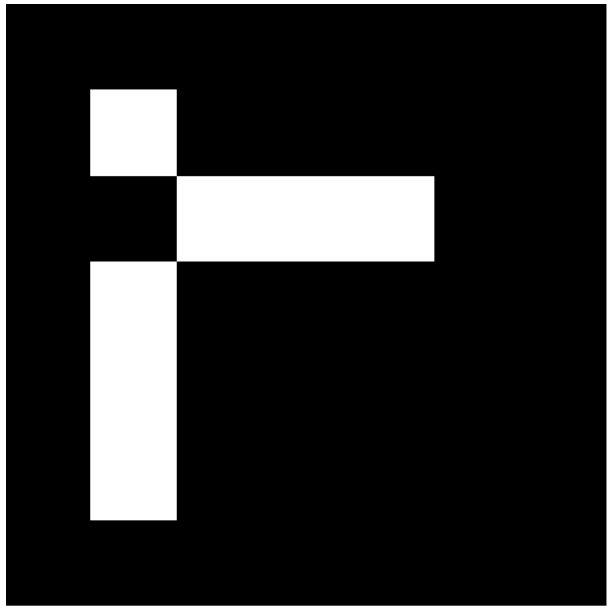
- Individuare e manipolare componenti su linea di assemblaggio
- Navigazione e movimentazione rapida
- Guida di bracci robotici

E. Lamon, M. Leonori, W. Kim and A. Ajoudani, "Towards an Intelligent Collaborative Robotic System for Mixed Case Palletizing," *2020 IEEE International Conference on Robotics and Automation (ICRA)*, Paris, France, 2020, pp. 9128-9134, doi: 10.1109/ICRA40945.2020.9196850.

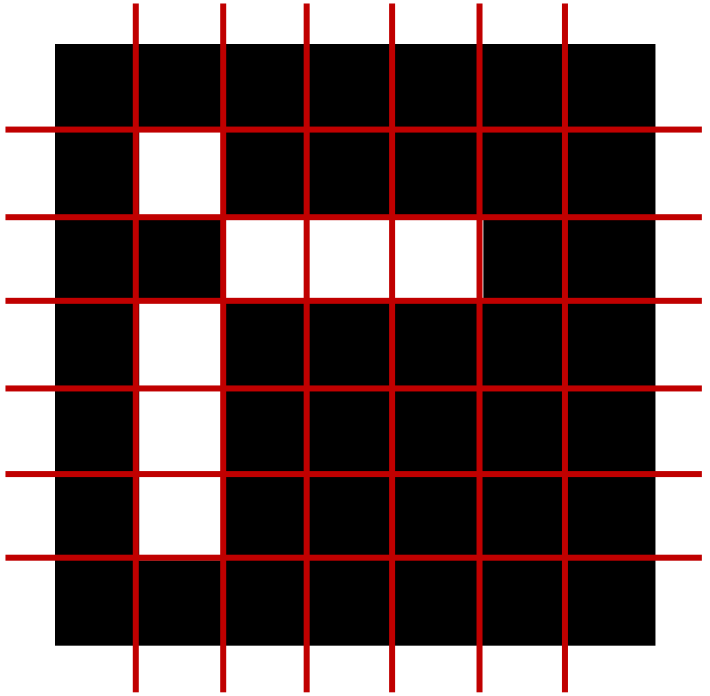
Cosa sono gli ArUco Marker

1. Marker quadrato composto da un ampio bordo nero e una matrice binaria interna che determina d'identificatore (ID) → simile a un barcode
2. Griglia 7x7 → matrice di uguale dimensione
3. Il dato dell'ArUco è visible dalla matrice interna (5x5)

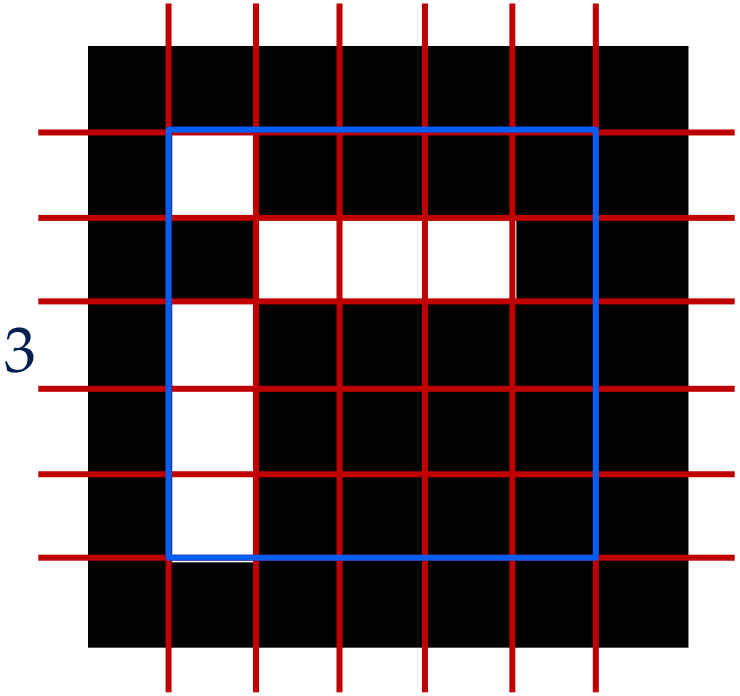
1



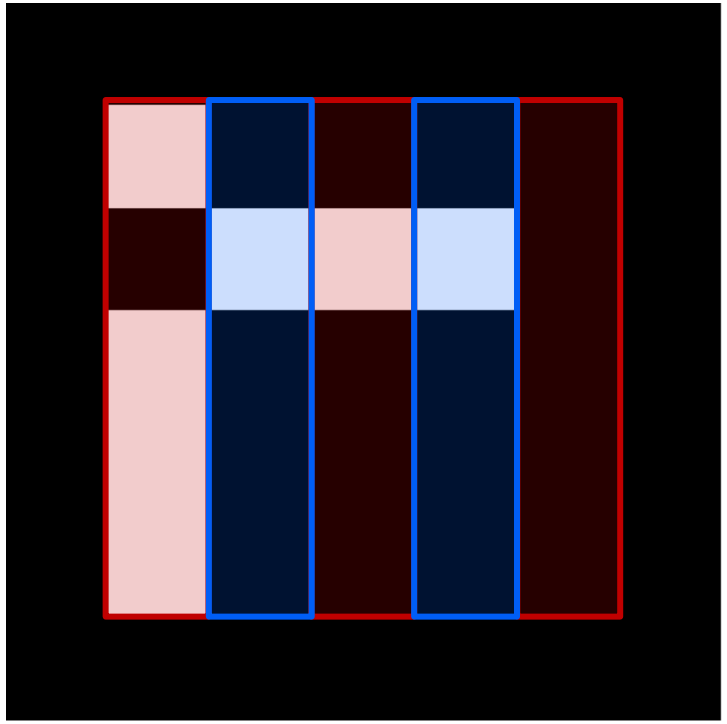
2



3



Cosa sono gli ArUco Marker



La griglia interna (5x5) presenta quadrati bianchi (1) o neri (0) → codice binario stampato

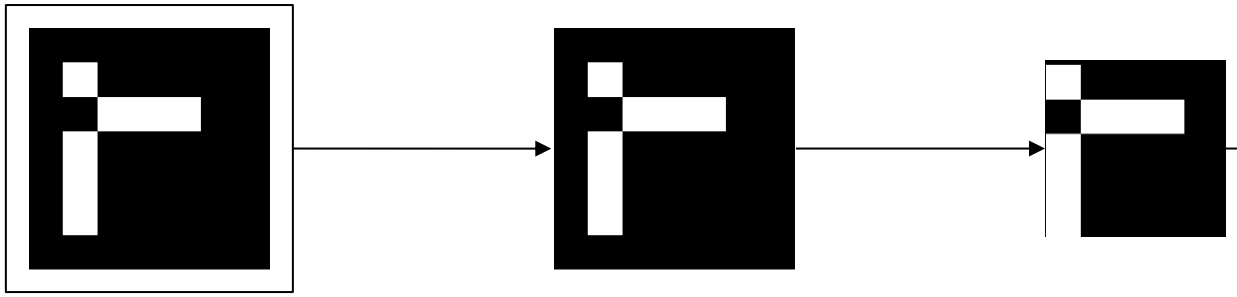
Nella griglia:

- La I, III, V colonna rappresentano i parity bits, usati per rilevare e correggere errori durante il riconoscimento del codice
- La II e IV colonna rappresentano i data bits, rappresentativi dell'informazione effettiva contenuta nel marker (ID univoco) → permettono di distinguere un marker dall'altro in un dizionario specifico

→ 10 data bits totali → il numero massimo di marker che può essere codificato è $2^{10} = 1024$, con ID compreso tra 0 e 1023

Tecnica di codifica

1. Estrarre l'ArUco dall'immagine
2. Rimuovere l'extra padding
3. Dividere l'immagine in una griglia 5x5 e vedere i codici della II e IV colonna dall'alto verso il basso, da sx a dx



1	0	0	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	0	0
1	0	0	0	0

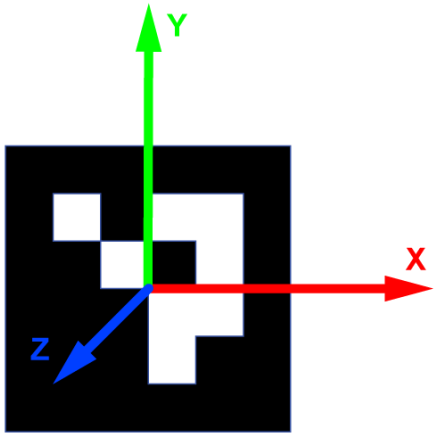
BIN: 00 11 00 00 00



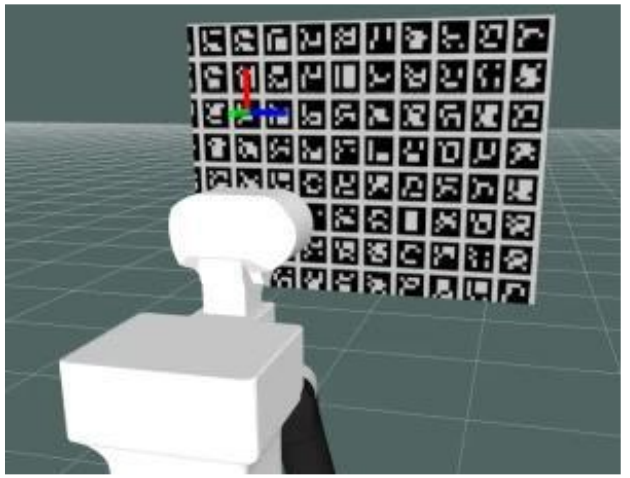
Marker ID: 192

Generare un ArUco:
https://chev.me/arucogen/

Benefici degli ArUco



1. **Rilevamento robusto**: Il bordo nero e il modello binario rendono i marcatori robusti alle varie condizioni di illuminazione, distorsioni e alle occlusioni parziali.
2. **Stima della posa**: I quattro angoli del marcatore forniscono corrispondenze sufficienti per stimare la posa della telecamera.
3. **Rilevamento e correzione degli errori**: Il modello binario consente l'applicazione di tecniche di rilevamento e correzione degli errori.



Generare gli ArUco Marker - OpenCV

Open CV = Open Computer Vision → libreria usata per applicazioni di image processing

- Fornisce dei dizionari predefiniti per dimensioni e numero di marker

```
pip3 install opencv-contrib-python==4.5.4.60 transforms3d==0.4.2
```

Generare ArUco → uso del modulo OpenCV cv2.aruco

Supponiamo di voler generare un marker 4x4 con 100 IDs unici

```
aruco_dict = cv2.aruco.getPredefinedDictionary(cv2.aruco.DICT_4x4_100)
```

Gli altri dizionari disponibili sono:
DICT_4X4_100, DICT_4X4_1000, DICT_4X4_250, DICT_4X4_50,
DICT_5X5_100, DICT_5X5_1000, DICT_5X5_250, DICT_5X5_50,
DICT_6X6_100, DICT_6X6_1000, DICT_6X6_250, DICT_6X6_50,
DICT_7X7_100, DICT_7X7_1000, DICT_7X7_250, DICT_7X7_50,
DICT_ARUCO_ORIGINAL

Dimensione NxN maggiore → maggiore robustezza del marker, perché contiene più bit e permette una migliore correzione degli errori

Numero di marker M in un dizionario maggiore → maggiore è il numero di ID → maggiore rischio di errori per marker simili

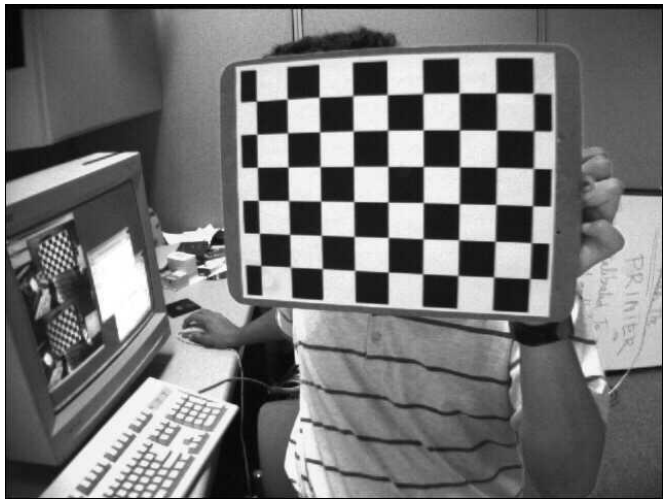
Individuare il marker - OpenCV

1. Caricare l'immagine con la funzione di OpenCV imread
2. Convertire l'immagine in una scala di grigi per migliorare il processo di detection
3. Detection del marker con la funzione detectMarkers

Stima della posa usando ArUco Marker

Determinare la posizione e l'orientamento della telecamera rispetto al marker:

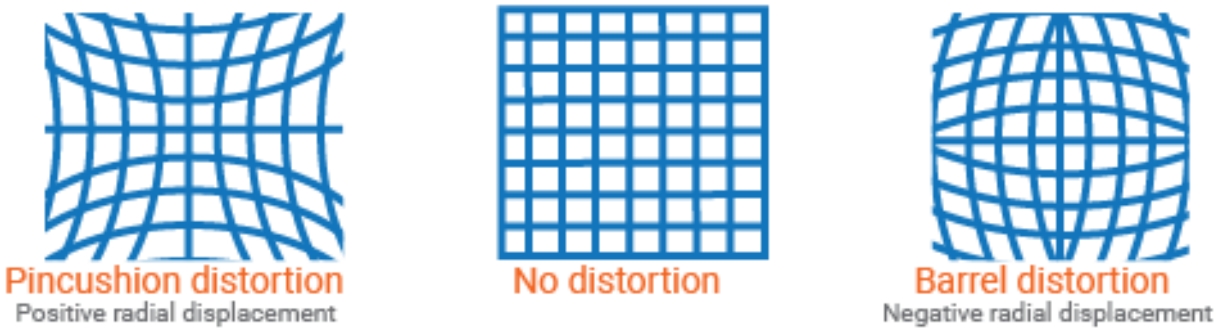
1. **Calibrazione della camera**: ottenere la matrice della camera e i coefficienti di distorsione
2. **Stima della posa**: usa la funzione estimatePoseSingleMarkers per stimare la posa dei marker individuati

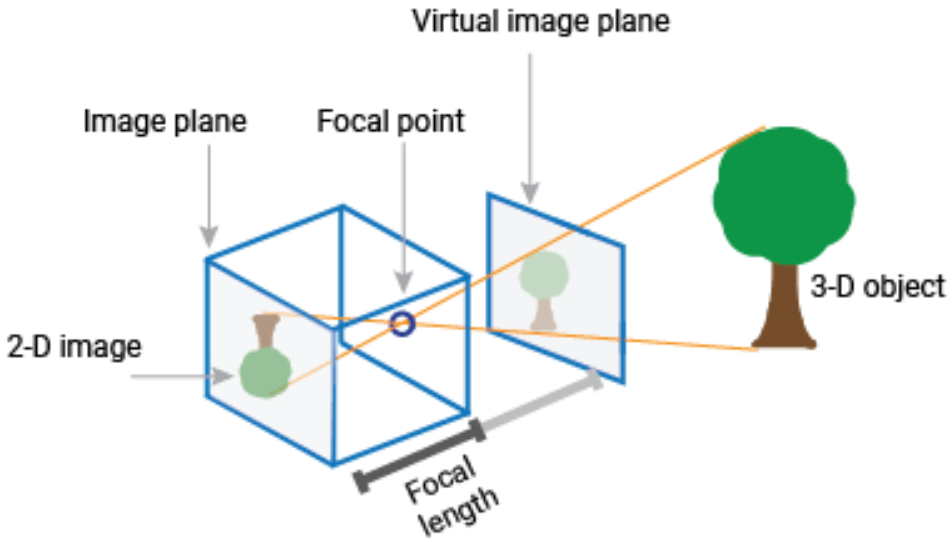


Calibrazione della camera → passaggio cruciale nella visione e nell'analisi delle immagini

- **Schema di calibrazione**: È un modello di scacchiera con quadrati bianchi e neri noti

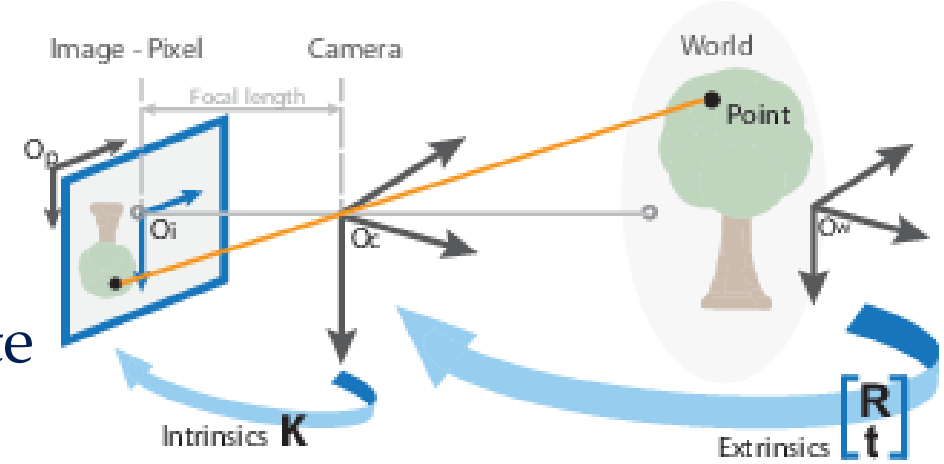
Correzione della distorsione → uso di **coefficienti di distorsione** per correggere le imperfezioni introdotte dalla fotocamera → proiezione più accurata dei punti 3D sulla superficie 2D dell'immagine



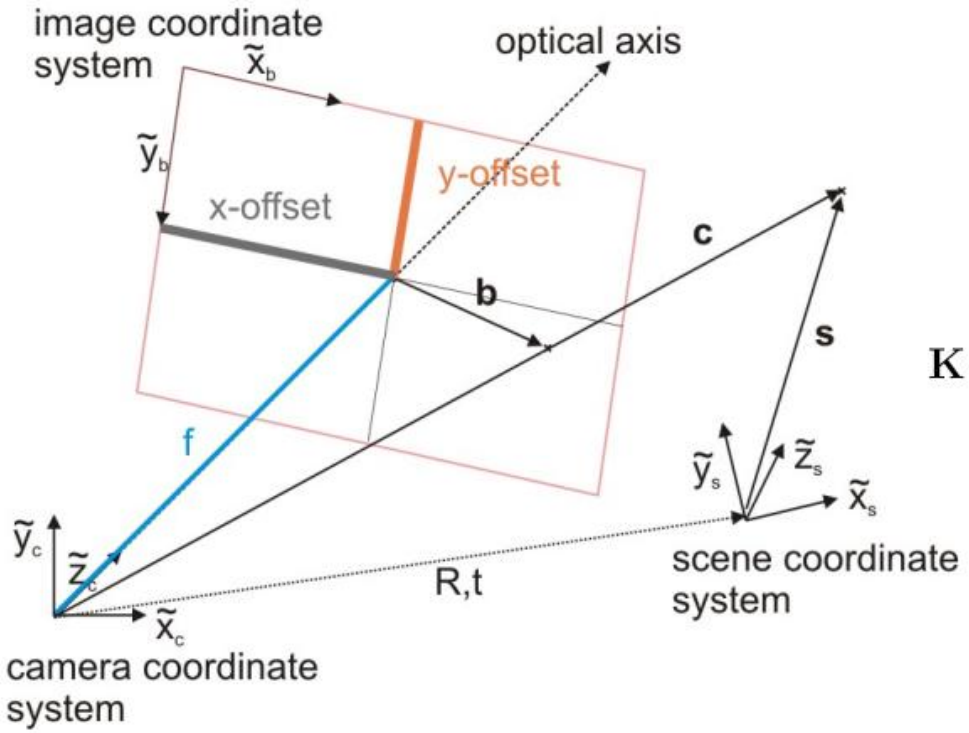


Pinhole: camera senza lenti e con una piccola apertura. La luce passa nell'apertura e proietta un'immagine invertita nella parte opposta della camera

Per andare dal 3D al 2D → **camera matrix**
Parametri intrinseci **K** → mappare le coordinate della camera nel piano dell'immagine



Calibrazione della camera



$$\mathrm{K} = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}$$

focal length — x-offset — y-offset

f_x, f_y: lunghezze focali lungo x e y (quanto è ingrandita l'immagine)

c_x, c_y : coordinate del centro ottico rispetto ad image coordinate system

$$\begin{pmatrix} \tilde{x}_b \\ \tilde{y}_b \\ 1 \end{pmatrix} = K \begin{pmatrix} \tilde{x}_c \\ \tilde{y}_c \\ \tilde{z}_c \end{pmatrix}$$

In **ROS2** → calibrazione sottoscrivendo al topic che termina .../camera_info:

camera_matrix= np.array(msg.k).reshape(3,3) → matrice 3x3 per la distorsione
dist_coeff = np.array(msg.d) → coefficienti di distorsione

Queste variabili sono in input a estimatePoseSingleMarkers

L'immagine si ottiene sottoscrivendo al topic che termina .../image_raw

La posa dell'ArUco è un messaggio del tipo geometry_msgs.msg.PoseStamped

Comandi utili

```
ros2 run rqt_image_view rqt_image_view
ros2 topic echo …/camera_info
ros2 topic info …/camera_info
ros2 interface show sensor_msgs/msg/CameraInfo
```